

CXL–GPU: Pushing GPU Memory Boundaries with the Integration of CXL Technologies

Donghyun Gouk^{ID}, Panmnesia, Inc., Daejeon, 34136, South Korea

Seungkwan Kang^{ID} and Seungjun Lee^{ID}, KAIST, Daejeon, 34141, South Korea

Jiseon Kim^{ID}, Kyungkuk Nam^{ID}, Eojin Ryu^{ID}, Sangwon Lee^{ID}, Dongpyung Kim^{ID}, Junhyeok Jang^{ID}, and Hanyeoreum Bae^{ID}, Panmnesia, Inc., Daejeon, 34136, South Korea

Myoungsoo Jung^{ID}, Panmnesia, Inc., Daejeon, 34136, South Korea, KAIST, Daejeon, 34141, South Korea, and Panmnesia, Inc., Seoul, South Korea

This work introduces a GPU storage expansion solution utilizing compute express link (CXL), featuring a novel GPU system design with multiple CXL root ports for integrating diverse storage media (dynamic random-access memory and/or solid-state drives). We developed and siliconized a custom CXL controller integrated at the hardware register transfer level (RTL), achieving two-digit nanosecond round-trip latency, the first in the field. This study also includes speculative read and deterministic store mechanisms to efficiently manage read and write operations to hide the endpoint's backend media latency variation. Performance evaluations reveal our approach significantly outperforms existing methods, marking a substantial advancement in GPU storage technology.

Large-scale deep learning models, such as large language models and mixtures of experts, have become prevalent across various computing domains. However, their memory demands often far exceed the capacity of accelerators like GPUs. For instance, while models with 1 billion parameters require approximately 16–24 GB of GPU memory for loading and training, models exceeding 100 billion parameters are increasingly commonplace.

To overcome these limitations, researchers have explored various approaches that leverage storage or external memory systems. One notable example

is GPUDirect storage (GPUDirect¹) which enables direct mapping of the GPU's PCIe base address register (BAR) to a target solid-state drive (SSD) by altering the storage stack. This approach facilitates the management of large-scale model parameters by utilizing the substantial capacity of SSDs. However, GPUDirect's adoption remains limited due to its complexity. Treating the SSD as a block device introduces challenges, including the management of the file system and mismatch in input–output (I/O) granularity. Furthermore, GPUDirect requires manual intervention for storage and memory operations, complicating the copy-then-execute programming model and deterring widespread usage.

In contrast, unified virtual memory (UVM²) provides a more straightforward solution by enabling shared virtual memory access for both CPU and GPU. This approach integrates GPU and host memory into a unified address space, allowing seamless data access. UVM automatically handles memory allocation and

0272-1732 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.

Digital Object Identifier 10.1109/MM.2025.3582433

Date of publication 7 July 2025; date of current version 24 December 2025.

migration based on demand, simplifying the management of large datasets. As a result, UVM has become a widely adopted technique for deep learning frameworks like TensorFlow and DGL.

In this study, we present the compute express link (CXL) as a transformative technology for GPU storage expansion. Figure 1(c) illustrates the high-level framework for the proposed GPU storage expansion approach. CXL leverages the PCIe physical data link to map devices, referred to as endpoints (EPs), to a cacheable memory space accessible by the host. This architecture allows compute units to directly access EPs using standard memory requests. Unlike JEDEC's double data rate (DDR)-standard dynamic random-access memory (DRAM) interfaces, which rely on synchronous communication, CXL enables asynchronous communication with compute units, akin to block storage. This flexibility supports EP implementation using diverse storage media such as nonvolatile memory (NVM)-based SSDs³ and DRAM.^{4,5}

Despite its potential, integrating CXL for GPU storage expansion presents a significant challenge due to the absence of a native CXL logic fabric and subsystem in GPUs to support DRAM/SSD EPs as memory expansion devices. To address this limitation, we have developed a CXL hardware layer stack and implemented it through a custom CXL controller designed at the hardware register transfer level (RTL), integrated within the CXL logic fabric. Additionally, we propose a GPU architecture featuring multiple CXL root ports equipped with these controllers, each capable of interfacing with DRAM/SSD EPs via a host bridge. This work represents the first demonstration of a silicon-based CXL controller achieving sub-two-digit nanosecond round-trip latency, signifying a major advancement in high-speed memory expansion technology.

We further enhanced the design of our CXL controller to improve GPU storage expansion performance. While achieving the fastest CXL round-trip latency to date, the storage media in EPs remains slower than local GPU memory. To mitigate this latency gap,

we propose two key strategies for GPU-EP interaction: speculative read (SR) and deterministic store (DS).

The SR mechanism leverages a module in our controller to manage load instructions, utilizing features from CXL 2.0. It anticipates target addresses by pre-sharing them with DRAM/SSD EPs, enabling them to prefetch target pages before the actual requests. This approach significantly reduces the latency impact of the storage media. To avoid overloading EPs with SR requests, we integrate the CXL quality of service (QoS) telemetry feature to monitor the EP's status and regulate SR traffic, ensuring a balance between performance and system load. For scenarios involving slower EP write performance, such as with NVM-based SSDs, the DS mechanism addresses the variability in write speeds. It performs concurrent writes to both GPU memory and the SSD EP, immediately completing store requests. Since GPU workloads are predominantly read-oriented, this "fire-and-forget" strategy is proven effective. However, in cases with intensive write activity or SSD tail latencies, the DS mechanism identifies the SSD EP as a bottleneck. In such instances, our controller temporarily buffers incoming data in GPU memory, deferring their transfer to the SSD EP. This allows the GPU to sustain the fire-and-forget approach for the majority of store operations, delivering improved performance across diverse workloads.

To evaluate the performance of our CXL hardware framework, we fabricated the CXL controller using state-of-the-art silicon technology. Integration with the GPU was achieved using Vortex,⁶ a RISC-V-based general-purpose GPU. The effectiveness of our GPU storage expansion approach was tested with custom-designed add-in-card (AIC) devices, as shown in Figure 1(d). Comprehensive testing was conducted using various storage media and GPU storage expansion techniques through RTL-level hardware simulations. Our GPU storage expansion approach significantly outperforms UVM and GPUDirect storage,⁷ achieving $50\times$ and $35.4\times$ higher performance, respectively.

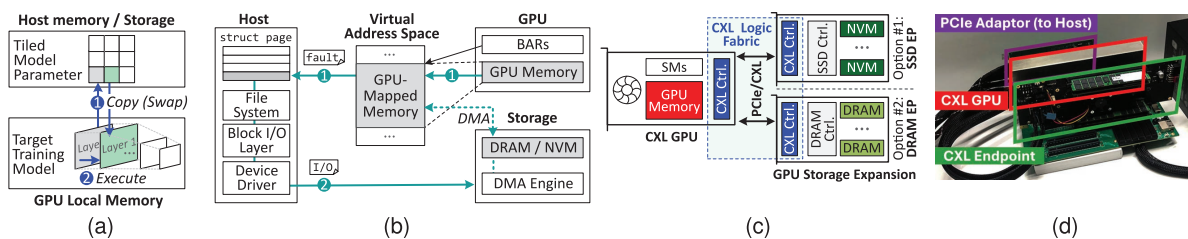


FIGURE 1. High-level view of GPU storage expansion. (a) Copy-then-execute. (b) GPU/Storage direct DMA. (c) Proposed CXL-based expansion. (d) AIC-level prototype.

MEMORY MANAGEMENT IN GPU AND CXL TECHNOLOGIES

Data Movement Management in GPUs

Copy-Then-Execute Model

GPU memory is crucial for storing not only the model parameters but also additional data such as gradients and intermediate computation buffers. The resulting memory requirement can be eight times larger than the parameters, exceeding GPU capacity. To address this limitation, GPUs often rely on host memory and storage, necessitating frequent data transfers between the CPU and GPU. Figure 1(a) illustrates the “copy-then-execute” model, commonly used for large-scale deep learning. In this model, parameters are divided into smaller segments or tiles, such as the model layers. This enables layer swapping ❶, allowing subsequent layers to process ❷. However, frequent data transfers create performance overhead and complicate GPU usage. Users must explicitly manage (tile) data structures and coordinate data migrations, which is cumbersome and inefficient, ultimately posing a barrier to GPU adoption.

Unified Virtual Memory

NVIDIA and AMD support UVM² allowing both CPU and GPU to access data in a unified virtual address space. When the GPU requests data not in memory, a page fault occurs, triggering a PCIe interrupt to the host. The host-side software then allocates a new page in GPU memory and transfers the required data. UVM also replaces older pages with new data, ensuring efficient memory use. These features have made UVM widely adopted in frameworks like TensorFlow. However, UVM can introduce performance bottlenecks due to the substantial latency of the host runtime resolving GPU’s page faults.

Direct Memory Access (DMA) for GPU and Storage

UVM faces limitations in memory capacity, as it relies on host-side DRAM. Alternative approaches expand GPU memory using block storage, significantly increasing memory capacity. Techniques such as GPUDirect¹ grant the storage’s DMA engine to access the GPU’s memory space, enabling direct data transfers between GPU and storage. As shown in Figure 1(b), they achieve this by mapping the GPU’s BAR to the virtual address space and creating a `struct page` for GPU memory within the host kernel. This configuration allows the host’s filesystem to share the GPU-mapped memory with storage, enabling DMA operations directly to

GPU memory. However, these approaches are limited as they rely solely on the storage DMA engine. Consequently, when the GPU encounters on-demand page faults ❶, the host runtime must translate them into storage I/O requests ❷, resulting in overheads comparable to UVM.

CXL Memory Expander

End-to-End Latency Analysis

Many industry vendors have developed prototypes or proofs-of-concept for CXL memory expanders that leverage CXL 2.0, but commercial products or in-depth latency information from an end-user perspective remains scarce.

CXL’s latency differs from JEDEC/DDR standards, which have precise timing specifications. Instead, CXL latency encompasses the full round-trip path across multiple layers. A memory request from the host CPU is converted into a CXL flit at the transaction layer, which supports the functionalities and interfaces required by each subprotocol. This flit progresses to the link layer, which manages communications such as buffering and acknowledgments. The flit is then passed to the flex bus physical layer, which sends it to the target EP based on link speed and lane configuration. For accurate management of CXL subprotocols, the EP must integrate these layers as well. After the EP’s layer stack, the flit returns to the system bus, reaching either a memory or an SSD controller. The data are then retrieved from the storage medium and returned to the host CPU’s system bus, traversing the hardware layers. The structure and function of this hardware layer stack play a critical role in determining roundtrip latency. Although market products are yet to be released, studies conducted by Samsung⁴ and Meta⁵ have reported latency of 250 ns for their prototypes.

CXL With an SSD Integration

The asynchronous nature of CXL.mem enables various memory types, including SSDs and different DRAM versions, providing flexibility in configurations. Industry proposals have explored integrating SSDs with CXL, including Intel’s planned release of a CXL-attached Optane SSD and Samsung’s introduction of the CXL-hybrid SSD (CMM-H). Although not yet commercialized, these SSDs are expected to use DRAM as a cache to mitigate slow storage media. Consequently, the performance will depend heavily on effective DRAM management. In addition, write operations on such devices are anticipated to be slower than reads and may occasionally encounter tail latency. For instance, flash memory necessitates garbage collection to reconcile

the mismatch between write and erase unit sizes. Therefore, SSD-CXL integration must carefully manage writes to ensure performance and reliability.

DESIGN OF CXL-INTEGRATED GPU

CXL Hardware Layer Stack

In response to the lack of a publicly accessible CXL hardware stack, we developed a series of hardware layers that support the CXL subprotocols, consolidating these into a unified controller. The controller is designed for CXL 3.1 while offering backward compatibility with earlier versions of CXL (2.0/1.1). The flex bus physical layer is integrated with our PCIe physical coding sublayer, supporting both PCIe and CXL layer stacks seamlessly over elastic buffers.⁷

To address the dual requirements of PCIe and CXL, particularly the power management and administrative operations, an arbitrator state machine has been incorporated into the controller. This state machine optimizes efficient resource allocation between PCIe and CXL tasks, ensuring performance and stability. The controller has undergone extensive testing, achieving a roundtrip latency in tens of nanoseconds, including the overhead of protocol conversion between standard memory operations and CXL flits. The controller has been integrated into hardware RTL implementations of GPU/CPU prototypes and memory expanders. Overall, our controller reduces latency by over three times compared to SMT4 and TPP.⁵

GPU Architecture Design and Integration

For the EP devices, we have integrated the CXL controller with memory and SSD controllers. This allows extending its back-end storage to the host system as host-managed device memory (HDM). The host is informed of this functionality through a feature analogous to PCIe capabilities, specifically adapted for

HDM usage. Integrating with GPU architectures, however, poses challenges because the EP device must interface with the GPU cache. To address this, we designed a specialized CXL root complex with a host bridge that includes multiple root ports. This configuration, shown in Figure 2(a), connects the host bridge to the system bus port and several CXL root ports. The core component, the HDM decoder, manages the allocation of system memory address ranges, referred to as host physical addresses (HPA), for each root port. When a GPU issues a memory request ❶, the corresponding root port translates the request into a CXL flit ❷ and forwards it to the CXL controller ❸. This design ensures seamless PU-storage communication, facilitating efficient data access.

System Configuration and Memory Space

Figure 2(a) shows the CXL root complex placement within our GPU architecture, implemented using the Vortex GPU framework and its hardware RTL.⁶ In the Vortex architecture, the GPU's computational units, known as streaming multiprocessors (SMs), connect to the system bus through the last-level cache. The system bus also interfaces with a GPU memory controller and a PCIe EP, which supports host communication and kernel execution. In our design, the CXL root complex is integrated into the system bus alongside a simplified core responsible for initializing the connected EPs, the host bridge's HDM decoder, and the HPAs of each root port. During initialization, firmware identifies CXL EPs by examining their configuration space and PCIe BARs and aggregates each EP's memory address space by analyzing the HDM capability registers. The firmware then records this information in the HDM decoder of the host bridge, specifying the base address and size of the HDM for each root port [cf. Figure 2(a)].

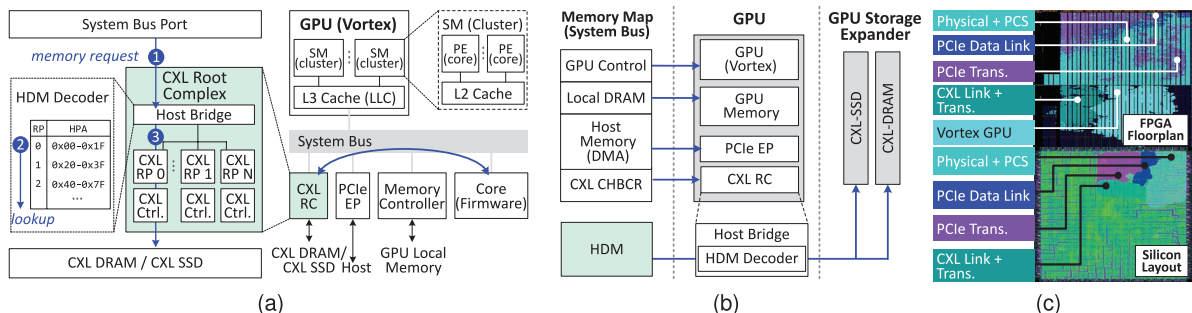


FIGURE 2. Design of CXL-integrated GPU. (a) GPU architecture. (b) GPU memory space. (c) Hardware prototypes.

After initialization, the GPU's system bus memory map is structured as shown in Figure 2(b). This map defines the allocation of address space across devices, segmented by function. For instance, when an SM generates a request targeting host memory, it interacts with the PCIe EP, which routes the request to the host. Similarly, GPU storage expanders under the CXL root complex are integrated into the system's memory architecture. Upon receiving a memory request for this system memory segment, the CXL root complex consults the HDM decoder, converts the request into a CXL flit, and forwards it to the corresponding root port and controller.

The systems employing our CXL controllers, including the Vortex GPU and EP devices, are realized on a 7-nm field-programmable gate array-based custom AIC. The associated RTL designs are detailed in Figure 2(c).

OPTIMIZATION OF CXL CONTROLLER FOR GPUS

The CXL round-trip latency depends on the back-end media type, such as DRAM or SSD. To address the latency challenges, we propose two optional strategies: 1) SR and 2) DS. SR aims to minimize read latency, while DS mitigates tail latencies during writes.

Accelerating Reads With Speculation

To enhance read performance, we employ the speculative read feature (*MemSpecRd*) introduced in CXL 2.0. This feature operates similarly to standard memory requests but provides hints about the addresses to be accessed shortly. Figure 3(a) shows our queue logic beneath the root port, consisting of two queues: the SR queue and the memory queue, each with 32 entries. Load requests are first added to the SR queue and processed into a *MemSpecRd* operation.

Implementation

To adapt *MemSpecRd*, we modify its address format. While originally designed for 64-B granularity, we repurpose the two least significant bits to indicate the length of the request, with the remaining bits specifying a 256-B memory offset, allowing requests to be coalesced. The requests in the SR queue are forwarded to the memory queue if available space exists. Once in the memory queue, requests are forwarded to the transaction layer. The SR reader module records the address and length of each issued request in a ring buffer. If a new request matches a previously issued SR request, it is directly forwarded to the memory queue. This allows

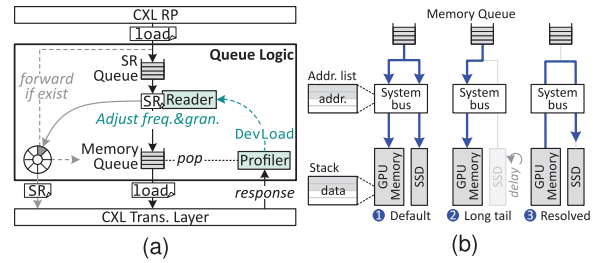


FIGURE 3. Optimization of CXL controller. (a) Speculative read. (b) Deterministic store.

the EP-side controller to preload data before the actual read requests.

When the EP's response arrives, the queue logic's profiler removes the completed request from the memory queue. Because SR requests also contribute to the back-end media bandwidth, the profiler keeps track of the EP's status by analyzing the QoS telemetry data of the response, specifically the *DevLoad* field.⁷ The *DevLoad* field, which reflects the workload of the EP, is shared with the SR reader to dynamically adjust the granularity of SR requests, optimizing system performance. The details are described next.

Load Control for Speculative Reads

The *DevLoad* field, defined as two bits, classifies the device's current load into four states: light load (*l1*), optimal load (*o1*), moderate overload (*mo*), and severe overload (*so*). When the device is in a *l1* state, the back-end media has sufficient bandwidth. In this case, the queue logic increases the granularity of SR requests by utilizing the modified *MemSpecRd* format, enabling efficient data preloading. In the *o1* state, the device operates at full bandwidth without being overwhelmed, so the current granularity is maintained. In contrast, a *mo* state indicates that the back-end media is experiencing increased traffic. To alleviate this, the granularity of SR requests is reduced to fetch only essential data. In a *so* state, the memory queue is completely saturated. The queue logic temporarily halts SR requests until the EP recovers and returns the *l1* state. This adaptive load control mechanism dynamically balances efficiency and stability, ensuring optimal performance across varying conditions.

Address Window Control

SR can improve the read performance of SSD-based EP devices by preloading data into the internal DRAM. However, because actual CXL requests operate at a 64-B granularity, preloading wrong data can cause

internal DRAM pollution and waste back-end media bandwidth. To mitigate this, the queue logic optimizes the address window for each SR request by analyzing the memory and SR queue. An address window consists of a starting and ending address. The queue logic begins with an initial address window starting and ending at the incoming SR request's address minus and plus its granularity, respectively. Since requests in the memory queue and SR queue each represent prior and future requests, the queue logic adjusts the window accordingly. Specifically, for each request in the memory and SR queue, the queue logic shifts the start and endpoint of the address window upwards and downwards by the same granularity, respectively. Finally, the computed address window is rounded to the nearest 256-B boundary. This ensures that SR requests target the most relevant data range, minimizing unnecessary data movement and optimizing DRAM utilization.

Deterministic Store for Precise Writes

The proposed DS technique conceals the slow write latencies by leveraging reserved space in the GPU's local memory.

Design and Implementation

As shown in Figure 3(b), when a write operation to an SSD is initiated, the request is concurrently sent to both the GPU memory and the SSD, allowing immediate release of the request ❶. If a delay of the SSD's response is observed, the queue logic temporarily stores the data from subsequent write request in the GPU memory's reserved address ❷. This temporary data storage is organized using a stack structure, designed to fill up during slow writes events and updated in place elsewhere. To maintain a record of each stack entry, an address list is stored within the system bus's internal SRAM. The stack is methodically emptied by flushing the data in the background ❸. Through the implementation of DS, the CXL root complex effectively shields the GPU from variations in write latency.

Fine Control for Internal Tasks

While the proposed hybrid memory management reduces write latency variability, internal SSD tasks such as garbage collection require finer control to mitigate their long tail latency. These tasks can temporarily make the SSD unavailable and cause congestion at the ingress port, degrading the performance of even read operations. To address this, we utilize the `DevLoad` field as well. Specifically, when an internal task is expected to reduce throughput temporarily, the

back-end media reports this through the `DevLoad` field beforehand. By monitoring `DevLoad`, our controller dynamically throttles write requests to the affected port and forwards them to GPU memory, ensuring that the throughput reduction does not lead to severe performance degradation.

EVALUATION

Methodology

Our hardware prototype provides detailed latency characteristics that are not accessible to end users but lacks flexibility to explore the design space for storage expansion. To address this, we developed a simulator that accurately models the behavior of our prototype. We performed an RTL behavior simulation of Vortex using Verilator⁸ and extracted data based on its performance counters and waveform dump. In addition, to add a back-end media, we connected DRAMSim³⁹ to the memory interface in the verilated RTL. To account for latency from PCIe/CXL/host runtime, we created a custom DRAMSim configuration that includes this additional latency. For PCIe and CXL bus latencies, we used real values measured from our application-specified integrated circuit. For UVM and GPUDirect's host runtime overhead, we accounted 45 μ s.¹⁰

Configurations

For the evaluation, we consider five different GPU configurations: 1) UVM, 2) GDS, 3) CXL, 4) CXL-SR, and 5) CXL-DS. UVM employs NVIDIA's unified virtual memory,² extending GPU memory using host DRAM. GDS uses GPUDirect storage,¹ enabling DMA between the GPU and storage. In both cases, data are transferred to the GPU per page through the host runtime. CXL represents an expander implemented with proposed controller. CXL-SR and CXL-DS build upon CXL by incorporating SR and DS techniques, respectively. We also evaluate an ideal configuration, GPU-DRAM, which assumes infinite on-device GPU memory. For SR and DS evaluations, we simulate various storage media as in Table 1(a).

Workloads

We evaluate eleven workloads from the Rodinia benchmark suite¹¹ and also evaluate two real-world workloads, `gnn` and `mri`. The `gnn` workload is composed of `bfs`, `vadd`, and `gemm`, while the `mri` workload is composed of `sort` and `conv3`. To ensure fair comparison across configurations, the input data sizes are adjusted to fit within the $10 \times$ bigger capacity of the GPU's local memory. For clarity, the workloads are categorized into

compute and memory-intensive applications, with the latter further divided into load and store-intensive subsets. All workloads are arranged in ascending order based on their memory access ratios. The important characteristics of the workloads are summarized in Table 1(b).

Performance Analysis

Figure 4(a) and (b) presents the performance of various GPU configurations with CXL memory expanders using DRAM and Z-NAND as back-end media, respectively. All execution times are normalized to the performance of the GPU-DRAM configuration.

DRAM-Based Expanders

Figure 4(a) compares UVM, CXL, and GPU-DRAM, as SR and DS mechanisms are only relevant for expanders with non-DRAM back-end media. As shown, UVM performs $52.7 \times$ worse than GPU-DRAM, with pronounced latency increases for load-intensive workloads such as *gemm*, *vadd*, and *saxpy*. These workloads involve sequential operations like vector addition, where data are read once and seldom accessed again. This access pattern generates numerous page faults, causing host runtime intervention. In contrast, CXL improves performance by $50 \times$ over UVM, by accessing the extended DRAM directly. The performance of CXL approaches that of GPU-DRAM, being slower by only 2.3%, 19.7%, and 6.8% for compute-intensive, load-intensive, and store-intensive applications, respectively. Note that the real-world applications show patterns similar to the operations that compose each application.

SSD-Based Expanders

Due to the significant performance variation, Figure 4(b) presents the results in a logarithmic scale. This analysis highlights SR and DS techniques' benefits. As shown, CXL-SR improves performance by $7.4 \times$ compared to CXL. This is due to the SR requests preloading data into internal DRAM, allowing sequential reads to be served immediately. However, the effectiveness of SR depends on applications exhibiting data access localities and avoiding excessive jumps across SR requests. For instance, 1-D vector computation (e.g., *vadd*, *saxpy*) and 2-D array computation workloads (e.g., *gemm*, *conv3*) exhibit the highest performance gains ($15.6 \times$) by issuing SR requests for upcoming computations. In contrast, graph-based applications with irregular access patterns, such as *path* and *bfs*, show limited benefits (67.6%). The internal DRAM can become crowded, leading to data eviction and reducing the effectiveness of SR.

TABLE 1. Evaluation setup. (a) Configs. (b) Workloads.

Vortex Configuration	Cores/Threads 8/8
PCIe Configuration	PCIe 5.0 (32GT/s) x8
CXL Configuration	PCIe 5.0 (32GT/s) x8 Sync-header bypass
DRAM	DDR5-5600[9]
Optane	Intel P5800X
Z-NAND	Samsung 983 ZET
NAND	Samsung 980 Pro

(a)

Workload Type	Acronym	Compute Ratio	Load Ratio
Compute Intensive	rsum	31.4%	53.3%
	stencil	37.5%	72.5%
	sort	38.1%	98.7%
Load Intensive	gemm	11.6%	99.9%
	vadd	15.6%	69.1%
	saxpy	16.2%	69.2%
	conv3	21.8%	78.6%
	path	27.0%	92.7%
Store Intensive	cfid	20.9%	42.6%
	gauss	23.5%	48.5%
	bfs	29.3%	43.2%
Real-World Workload	gnn	27.4%	73.8%
	mri	29.2%	53.3%

(b)

CXL-DS improves performance by 20.9%, 8.7%, and 62.8% over CXL-SR for each category. These benefits are particularly pronounced in store-intensive applications, as DS mitigates tail latency caused by internal SSD operations, preventing GPU threads from stalling.

Backend Media Latency Mitigation

Figure 4(c) illustrates the performance benefits of the SR and DS mechanisms across various back-end media, specifically Optane (O), Z-NAND (Z), and standard NAND (N). We analyzed workloads with a sequential access pattern (*vadd*), a random and load-intensive pattern (*path*), and a random and store-intensive pattern (*bfs*). For sequential workloads, SR can provide near-DRAM performance by hiding the latency of the back-end media. For instance, *vadd* achieves $21.5 \times$ performance improvement compared to CXL when using SR. DS provides up to a $4 \times$ performance gain for store-intensive applications (*bfs*) by hiding the tail latency during media management (e.g., garbage collection). The detailed analysis of each mechanism is described next.

Speculative Read

Figure 4(d) illustrates the performance benefits of the proposed SR mechanism with Z-NAND-based memory expansion. CXL-NAIVE generates 64-B MemSpecRd requests for all incoming requests. In contrast, CXL-DYN coalesces multiple SR requests using our flexible address format. CXL-SR builds on CXL-DYN by analyzing the SR and memory queues to adjust the SR address window. For comparison, the workloads were categorized by memory access patterns. Seq and Rand represent sequential and random memory accesses, respectively. Around represents access patterns where the direction of the sequential access alters in runtime. For instance, Gaussian elimination (*gauss*) accesses the current or previous row based on the row's first element.

CXL-NAIVE increases performance $1.9 \times$ compared to CXL. This benefit occurs because data can be read from NAND in advance by sending MemSpecRd requests for requests that are waiting in the GPU's memory queue. This approach prevents cold misses in the SSD's internal DRAM for all workloads. Specifically, under CXL, the Seq, Around, and Rand workloads reach DRAM hit rates of 47.4%, 31.2%, and 10%, respectively, while under CXL-NAIVE, they reach 88.4%, 56%, and 32.1%, respectively.

However, CXL-NAIVE remains $6.6 \times$ slower than GPU-DRAM due to the limited coverage of the SR requests. CXL-DYN achieves an additional performance gain of $4.5 \times$ for Seq workloads by providing the opportunity to preload data more data promptly. This is because the aggressive preloading raises the SSD DRAM hit rate for Seq workloads to over 99%. Around and Rand workloads gain less due to the probabilistic utilization of the preloaded data, resulting in 10.6% and 5.7% improvement,

respectively. For Rand workloads, simply increasing the granularity of preloading can lead to SSD DRAM pollution and reduce performance. In practice, sending the largest possible SR requests at all times lowers the SSD DRAM hit rate for Rand from 32.1% to 30.7%. However, since CXL-DYN can dynamically adjust the granularity of MemSpecRd requests, it achieves a higher hit rate (34%) for Rand.

For Around workloads, which generally follow sequential access patterns, the SSD DRAM hit rate is higher than CXL-NAIVE, but it still remains at 57.4% because the next access may occur either before or after the current address. CXL-SR overcomes this limitation by dynamically adjusting the address window of the SR requests, yielding an additional $2.1 \times$ performance improvement for Around. This is possible because CXL-SR can analyze the incoming addresses and decide whether to send MemSpecRd requests for addresses before or after the current one, raising the SSD DRAM hit rate for Around to 75.8%.

Deterministic Store

We evaluated two Z-NAND-based systems, each with (CXL-DS) and without (CXL-SR) our proposed DS mechanism, under long-tail cases due to garbage collection. Figure 4(e) shows the time series of load/store latencies and ingress queue occupancy. CXL-SR experiences higher load/store latencies as store requests saturate the ingress queue. The GC process even recurs shortly after completion, as the accumulated store requests flood into the media, depleting the free pages. In contrast, CXL-DS avoids sending store requests to the EP. This strategy prevents ingress queue congestion,

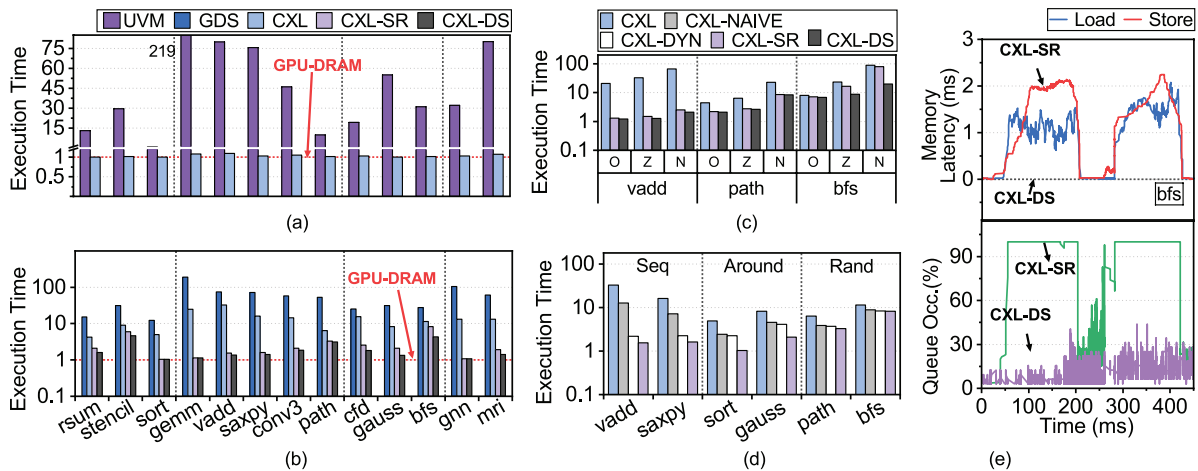


FIGURE 4. Performance analysis. (a) Overall (DRAM). (b) Overall (Z-NAND). (c) Backend media sensitivity. (d) Speculative read. (e) Deterministic store.

hiding tail latency of both load and store requests, and prevents subsequent GC.

CONCLUSION

We introduce a method to increase GPU storage using CXL technology, integrating DRAM and/or SSDs through multiple CXL root ports. Our custom-designed CXL controller, implemented directly in the hardware, achieves fast response times not seen before in this field. We further refine this system with features designed to improve both read and write operations to storage media. Our tests show that this approach improves upon existing solutions, offering a significant step forward in GPU storage capacity and efficiency.

ACKNOWLEDGMENTS

The work was supported in part by IITP under Grants RS-2023-00221040, RS-2024-00460762, RS-2025-02214652, RS-2025-02214654, and RS-2025-02263080, KIAT under Grants P0027923 and P0028225, and the MSS Technology Development Program under Grant RS-2023-00303967. The authors thank anonymous reviewers for their constructive feedback. Authors associated with KAIST contributed to introducing the concept of speculative read and deterministic store, with all remaining work and contributions brought by Panmnnesia. This work is protected by one or more patents.

REFERENCES

1. "GPUdirect storage." NVIDIA. Accessed: Feb. 26, 2025. [Online]. Available: <https://docs.nvidia.com/gpudirect-storage/>
2. "CUDA toolkit." NVIDIA. Accessed: Feb. 26, 2025. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-driver-api/>
3. M. Jung, "Hello bytes, bye blocks: PCIe storage meets compute express link for memory expansion (CXL-SSD)," in *Proc. 14th ACM Workshop Hot Topics Storage File Syst. (HotStorage)*, 2022, pp. 45–51, doi: [10.1145/3538643.3539745](https://doi.org/10.1145/3538643.3539745).
4. K. Kim et al., "SMT: Software-defined memory tiering for heterogeneous computing systems with CXL memory expander," *IEEE Micro*, vol. 43, no. 2, pp. 20–29, Mar./Apr. 2023, doi: [10.1109/MM.2023.3240774](https://doi.org/10.1109/MM.2023.3240774).
5. H. A. Maruf et al., "TPP: Transparent page placement for CXL-enabled tiered-memory," in *Proc. 28th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst. (ASPLOS)*, 2023, vol. 3, pp. 742–755, doi: [10.1145/3582016.3582063](https://doi.org/10.1145/3582016.3582063).
6. B. Tine, K. P. Yalamarthy, F. Elsabbagh, and K. Hyesoon, "Vortex: Extending the RISC-V ISA for GPGPU and 3D-graphics," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2021, pp. 754–766, doi: [10.1145/3466752.3480128](https://doi.org/10.1145/3466752.3480128).
7. "CXL specification revision 3.1" Compute Express Link, Beaverton, OR, USA, Aug. 2023. Accessed: Feb. 26, 2025. [Online]. Available: <https://computeexpresslink.org/wp-content/uploads/2024/02/CXL-3.1-Specification.pdf>
8. W. Snyder, P. Wasson, and D. Galbi, "Verilator." Veripool. Accessed: Feb. 26, 2025. [Online]. Available: <https://www.verilator.org>
9. S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "DRAMsim3: A cycle-accurate, thermal-capable DRAM simulator," *IEEE Comput. Archit. Lett.*, vol. 19, no. 2, pp. 106–109, Jul./Dec. 2020, doi: [10.1109/LCA.2020.2973991](https://doi.org/10.1109/LCA.2020.2973991).
10. T. Allen and R. Ge, "Demystifying GPU UVM cost with deep runtime and workload analysis," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, 2021, pp. 141–150, doi: [10.1109/IPDPS49936.2021.00023](https://doi.org/10.1109/IPDPS49936.2021.00023).
11. S. Che et al., "Rodinia: A benchmark suite for heterogeneous computing," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, Austin, TX, USA, 2009, pp. 44–54, doi: [10.1109/IISWC.2009.5306797](https://doi.org/10.1109/IISWC.2009.5306797).

DONGHYUN GOUK is the chief architect officer at Panmnnesia, Inc, Daejeon, 34136, South Korea. His research interests include computing switch/networks, artificial intelligence infrastructures, RISC-V, and link solutions including CXL, UALink, and Ethernet. Contact him at donghyun@panmnnesia.com.

SEUNGKWAN KANG is working toward his Ph.D. degree at KAIST, Daejeon, 34141, South Korea. His research interests include storage-integrated accelerators. Kang received his B.S. degree in electrical engineering from KAIST. Contact him at kangsk@camelab.org.

SEUNGJUN LEE is working toward his Ph.D. degree at KAIST, Daejeon, 34141, South Korea. His research interests include OS and storage stack optimization. Lee received his B.S. degree in electrical engineering from KAIST. Contact him at sjlee@camelab.org.

JISEON KIM is an assistant director at Panmnnesia, Inc, Daejeon, 34136, South Korea. Her research interests include edge computing, on-device artificial intelligence, memory, and link solutions including CXL, UALink, and Ethernet. Contact her at jskim@panmnnesia.com.

KYUNGKUK NAM is a senior professional at Panmnesia, Inc, Daejeon, 34136, South Korea. His research interests include ML acceleration, computing switch/networks, artificial intelligence infrastructures, and link solutions including CXL, UALink, and Ethernet. Contact him at kknam@panmnesia.com.

EOJIN RYU is an assistant director at Panmnesia, Inc, Daejeon, 34136, South Korea. His research interests include GPU/SSD integration, edge computing, and link solutions including CXL, UALink, and Ethernet. Contact him at neo21top@camelab.org.

SANGWON LEE is a director at Panmnesia, Inc, Daejeon, 34136, South Korea. His research interests include persistent memory, RISC-V, and link solutions including CXL, UALink, and Ethernet. Lee received his Ph.D. degree from KAIST. Contact him at swon@panmnesia.com.

DONGPYUNG KIM is an assistant director at Panmnesia, Inc, Daejeon, 34136, South Korea. His research interests include GPU/SSD integration, edge computing, and link solutions including CXL, UALink, and Ethernet. Contact him at dpkim@panmnesia.com.

JUNHYEOK JANG is a director at Panmnesia, Inc, Daejeon, 34136, South Korea. His research interests include machine learning, NVMe, reliability, security, and link solutions including CXL, UALink, and Ethernet. Contact him at jhjang@camelab.org.

HANYEOREUM BAE is a director at Panmnesia, Inc, Daejeon, 34136, South Korea. His research interests include new storage, ZNS, and link solutions including CXL, UALink, and Ethernet. Contact him at hyr.bae@camelab.org.

MYOUNGSOO JUNG is a full professor in the School of Electrical Engineering, KAIST, Daejeon, 34141, South Korea, and the CEO of Panmnesia, Inc, Daejeon, 34136, South Korea. His research interests include computer architecture, operating systems, storage systems, nonvolatile memory, parallel processing, heterogeneous computing, edge computing, artificial intelligence infrastructure, and link solutions including CXL, UALink, and Ethernet. He is the corresponding author of this article. Jung received his Ph.D. degree in computer science and engineering from The Pennsylvania State University. Contact him at mj@panmnesia.com.



IEEE COMPUTER SOCIETY
Call for Papers

It's Author's Choice!
Publish open access or
traditionally based on
your unique needs.

 **GET PUBLISHED**
www.computer.org/cfp

 IEEE
COMPUTER
SOCIETY  IEEE